ELI — Architecture

ELI v2.5.35

November 2026

Contents

Blueprint — ELI Architecture **1**

0. Design principles . . . 2
1. Repository map . . . 2
2. Entry points and launch . . . 3
3. The request lifecycle . . . 4
4. Routing layer (`eli/execution`) . . . 4
5–6. Agent bus and agents (`eli/cognition/agent_bus.py`) . . . 5
7. The orchestrator (`eli/cognition/orchestrator.py`) . . . 6
8. Inference layer (`eli/cognition`) . . . 6
9. Grounding and anti-confabulation (`eli/runtime`, `eli/core/grounding.py`) . . . 7
10. Execution layer (`eli/execution/executor_enhanced.py`) . . . 7
11. Memory subsystem (`eli/memory`) . . . 8
12. Perception (`eli/perception`) . . . 8
13. Persona (`eli/cognition/persona*`) . . . 8
14. Daemons and background loops . . . 9
15. Learning and self-training (`eli/learning`) . . . 9
16. EliWorld (`eli/world`, the EliWorld tab, `kernel/world_model.py`) . . . 9
17. Core infrastructure (`eli/core`) . . . 9
18. GUI (`eli/gui`) . . . 10
19. On-disk layout (`artifacts/`, runtime) . . . 10
20. Known seams and weak points . . . 10
21. Where to make a change . . . 10

Plugins, agent specifications and training surfaces . . . 11

> **Updated for v2.4.84 (September 2026).** Every CHAT turn runs the gradient orchestrator at a depth scaled to the reasoning mode, with the specialist bus composed inside it after shared retrieval (`eli/memory/retrieval.py`). Canonical S01–S12 tracing is in `eli/kernel/pipeline_trace.py`; stage 12 learning runs through `learning_coordinator.py`.

Blueprint — ELI Architecture

The structural map of ELI v2. File paths, counts and names below were checked against the source tree on 2026-09-25; where something is only true at runtime it is marked *(runtime)*.

> ELI is a **local-first, offline-by-default, model-agnostic** cognitive runtime and assistant GUI, and also a self-hosted web app (`api/server.py`, described in `ELI_USER_MANUAL.md`). No cloud on the inference path, no hardcoded model. **195,242 lines across 431 Python files in `eli/`**, plus `api/server.py` (2,292 lines). 230 capabilities in `capability_manifest.json`, 189 of them routable and 207 in the executor's supported list.

---

0. Design principles

1. **Local-first, offline-by-default.** Outbound network is refused at the socket boundary (`eli/core/netguard.py`) unless the owner enables it; features that need the network opt in.
2. **Model-agnostic.** No model name or size is hardcoded on the inference path. ELI loads whatever GGUF is configured and identifies its family from file metadata.
3. **Grounded.** A dedicated subsystem (§9) exists to stop a local model stating things it cannot back up.
4. **Persona is not scripted away.** ELI has a distinct voice; functional bugs are fixed without flattening it.

---

1. Repository map

Files and lines per package under `eli/`, measured 2026-09-25.

Package	Lines	Files	Role
`eli/runtime`	35,375	95	Grounding spine, evidence, response and introspection surfaces, daemons, self-improvement
`eli/gui`	27,447	27	PySide6 desktop app, panels, tabs, startup and first-boot wizard, animated face
`eli/execution`	26,293	13	Router and executor: intent → action → side effects
`eli/cognition`	20,824	47	Agent bus, 12-stage orchestrator, inference, persona, reasoning modes, tone and emotion
`eli/kernel`	17,545	8	`CognitiveEngine`, pipeline tracing, scheduler, self-upgrade, world model
`eli/core`	11,222	30	netguard, paths, settings, hardware profile, model download, full control
`eli/perception`	10,220	23	Vision, STT, TTS, wake word, voice tone, OS control, gaze
`eli/memory`	8,770	11	SQLite, FTS5, FAISS, knowledge graph, storage policy

Package	Lines	Files	Role
`eli/tools`	5,969	25	Image engine, news, capability registry, document tools
`eli/plugins`	5,911	34	Plugin manager, marketplace client, bundled plugins
`eli/learning`	4,251	14	LoRA self-training pipeline
`eli/planning`	4,000	19	Proactive daemon, habit scheduler, job, proposal and attention queues
`eli/integrations`	3,339	18	Ollama client, MCP client and server, cross-platform media, MPRIS
`eli/coding`	2,104	12	`CodeAgent`: plan → search → verify → repair
`eli/setup`	1,935	9	Installer, hardware policy, first-run wizard
`eli/utils`	1,838	5	Logging, platform compatibility
`eli/world`	1,793	26	World event bus, local world bridge, avatar and ontology
`eli/system`	1,190	3	Portable app control, process guard
`eli/contracts`	787	3	Typed runtime-status contracts
`eli/onboarding`	709	2	Onboarding interview
`eli/cli`, top level	236	4	Headless REPL, package entry
total	**194,736**	**429**	

The largest files: `eli/kernel/engine.py` (16,014 lines), `eli/execution/executor_enhanced.py` (15,845), `eli/gui/eli_pro_audio_gui_v2_0.py` (13,107), `eli/execution/router_enhanced.py` (8,271), `eli/gui/labs_tab.py` (5,744), `eli/memory/memory.py` (5,677), `eli/cognition/agent_bus.py` (3,517), `eli/runtime/deterministic_grounding_ga` (3,367), `eli/cognition/gguf_inference.py` (3,234).

---

2. Entry points and launch

Command	Target	Notes
`eli`, `eli-v2.0`, `eli-cli`	`eli.gui.app:main`	GUI (default)
`eli-mcp`	`eli.integrations.mcp.server:main`	ELI as an MCP server
`eli-jobs`	`eli.planning.jobqueue_cli:main`	durable job queue CLI

Command	Target	Notes
`python -m eli`	`eli/__main__.py`	GUI, or headless with flags
`python -m eli --headless`	`eli.cli.headless`	terminal REPL, no Qt
`./eli.sh`	`python -m eli`	venv launcher
`bash install.sh`		venv, dependencies, clean config seed, verification

Boot path (GUI): `eli/gui/app.py` → first-boot wizard when no model is configured (`eli/gui/panels/startup.py`) → `eli_pro_audio_gui_v2_0.py:main()` builds `EliMainWindow` → constructs the `CognitiveEngine` singleton → loads the GGUF named in `config/settings.json` → starts the daemons.

---

3. The request lifecycle

Every request goes through `CognitiveEngine.process()` (`eli/kernel/engine.py`).

```
user input
   │
   ▼
[Router]  eli/execution/router_enhanced.py :: route()
   │   priority pipeline → {action, args, confidence, meta.matched_by}
   ▼
CognitiveEngine.process(action, args, ...)
   │
   ├─ deterministic fast path (OS, media, status, job actions)
   │      → execute_action() → returned verbatim, no model
   │
   ├─ CHAT (any reasoning mode, Quick to Expert)
   │      → AgentOrchestrator.run() at mode depth
   │         S05 planner → shared retrieval → dispatch_specialists()
   │         → context assembly → broker.infer() → output governor
   │
   ├─ non-CHAT action
   │      → AgentOrchestrator → bus + ReAct tool loop
   │
   └─ orchestrator returns None or raises → AgentBus.dispatch() fallback
```

The canonical stage names (`pipeline_trace.STAGE_NAMES`) are S01 PERCEIVE_INGEST, S02 INPUT_GUARDS, S03 ROUTER, S04 GROUNDING_GATE, S05 PLANNER, S06 AGENT_BUS, S07 CONTEXT_ASSEMBLY, S08 INFERENCE_BROKER, S09 REASONING_SYNTHESIS, S10 OUTPUT_GOVERNOR, S11 RESPONSE_DELIVERY, S12 LEARNING_STATE_UPDATE. Enable tracing with `ELI_PIPELINE_TRACE=1` or `scripts/eli_startup.sh --trace`.

---

4. Routing layer (`eli/execution`)

- **`router_enhanced.py`**: `route(text) -> {action, args, confidence, meta}`. A regex-first priority pipeline of ordered prepasses (realtime web lookup, media, file and PDF, memory, control), then a core router, with an LLM intent fallback (`cognition/llm_intent.py`).
- **`execution_planner.py`**: typed `ExecutionPlan` and `RouteDecision` artifacts.
- **`route_contracts.py`, `operator_policy.py`, `shell_gate.py`**: what may be routed and executed.
- **`portable_intent_contract.py`, `media_runtime.py`, `operator_actions.py`**: media and OS intent wiring. Plugin actions are handled directly in `executor_enhanced.py`.

A command is carried out from a sentence that gives it. The last stages of the pipeline match on words wherever they stand, and used to carry commands out from inside sentences that were not telling ELI to do anything: a complaint that named Spotify stopped the film that was playing. Three pieces of form now decide, none of them a list of phrases:

- `runtime/command_splitter.py` reads a message as sentences and says, for each, what it tells ELI to do, or nothing: a sentence that starts with who or what it is about ("you did not open spotify", "it stopped again") or with a question word is about something; one framed as a request ("could you …", "i'd like you to …") or starting with what to do is an instruction. A sentence that turns into one ("this is great, play it again"; in speech, with no punctuation, "that was great can you play some jazz") is read as its two parts. It also holds the one list of verbs a command starts with (`COMMAND_VERBS`).
- `route_contracts.changes_something(action)` says, from the action's name, whether carrying it out changes anything. `route_contracts.fits(text, intent)` is asked of those that do: the sentence has to be an instruction, and beyond the command's own words it has to say little. The router asks it of the word-matching stages; the engine asks it of whatever the intent model resolves. A candidate that does not fit is passed over and recorded (`router.not_an_instruction`), and that turn is not sent to the intent model.
- A message of commands and talk ("see, you can do it! open spotify") is routed as `message.instructions_and_talk`: the engine carries the commands out through the executor and answers the rest through the full pipeline, with the result in front of the model and told that nothing else ran (`_carry_out_agreed`, the path a "yes" to an offer takes).

What a command names is what it acts on. `portable_intent_contract.web_open_target` and `window_target` are the one place each for an "open …" target that is a website and for the window a command names. A site is opened by an address that was given, is in the one table of site names (`app_aliases.WEBSITE_ALIASES`), or is found when the command runs (`runtime/site_address.py`: the conversation first, then a web search whose result carries the name); "that website" and "the second link" are looked for in the conversation. An address is never built from the words ("go to the youtube tab" used to open `https://the youtube tab`). Maximise, minimise, focus and tab commands keep the window they name; a bare one acts on the window in front. Every control in the catalogue's "App & window control", "Input control" and "Media" groups goes straight to the executor, read from the catalogue and not from a list.

Routes that were once bugs are pinned by tests and by `tools/eval/cases.yaml`: media controls need whole-word, command-shaped input; a bare "do a search" needs a subject; meta-questions stay in CHAT; realtime facts go to WEB_SEARCH; code words such as "repo" match whole words only, so "morning report" is not a codebase audit.

5–6. Agent bus and agents (`eli/cognition/agent_bus.py`)

`AgentBus.dispatch(user_input, intent, ...) -> DispatchResult` selects an agent set (`_select_agents_for_intent`, or a broad fan-out), runs it over a dependency DAG (flat parallel if the DAG fails) and aggregates. `DispatchResult` carries `agent_results`, `action_result`, `memory_context`, `aggregated_confidence`, `grounding_confidence` (agent evidence only), `agents_used`, `confidence_label` and `orchestrator_plan`.

15 registered agents (`_ALL_AGENTS`):

Agent	Role
`BusMemoryAgent`	recall from SQLite, FTS and FAISS into context
`SystemAgent`	runs executor actions (WEB_SEARCH, RUNTIME_AUDIT, …)
`HabitAgent`	learned habits
`SelfImprovementAgent`	recent failures, improvement proposals, corrections

Agent	Role
`ProactiveAgent`	proactive patterns and signals
`FrontierAgent`	frontier reasoning hooks
`PluginAgent`	dispatches to registered plugins
`CapabilityAgent`	what ELI can do (capability manifest)
`VoiceAgent`	STT and TTS engine state
`OrchestratorAgent`	plans and coordinates grounded synthesis
`FileCodeAgent`	searches the codebase for code-grounded answers
`ReflectionAgent`	session reflection
`IntrospectionBusAgent`	live runtime and self introspection
`KnowledgeGraphAgent`	entities and relations from the knowledge graph
`CriticAgent`	second-tier verifier; runs after its retriever dependencies

`SpecAgent` runs user-defined agent specifications against the local model. The coding pipeline, `CodeAgent` (`eli/coding/agent.py`, plan → search → verify → repair), is separate from the bus and is invoked for CODE_SOLVE and GENERATE_SCRIPT.

---

7. The orchestrator (`eli/cognition/orchestrator.py`)

Runs for all CHAT modes at gradient depth, and for non-CHAT actions. Shared retrieval lives in `eli/memory/retrieval.py` (`retrieve_for_turn()`), called by both `BusMemoryAgent` and the orchestrator so a turn is not searched twice:

- **Turn cache**: 8 seconds per process. Its key includes every retrieval limit, so a deeper pass with larger limits is never served the shallower result.
- **Time window first**: when the question names a period ("last week", "the past two weeks"), `query_planner.parse_window` produces a window. Memories inside it are fetched by date (`Memory.memories_between`) before any ranking, then ranked by topic. The search reports what the window did (`memory_diag`).
- **Sequential by design**: the llama.cpp embedder is not thread-safe.
- **Heuristic rerank** (`cognition/reranker.py`): lexical overlap × recency × importance. A neural cross-encoder is not in use.
- **FAISS tombstones**: `vector_store.mark_memory_deleted()` marks stale vectors without a rebuild; `compact_tombstones()` reclaims space.

Mode scaling (`reasoning_modes.py`): Quick uses the fast planner and a lean specialist profile; Normal, Advanced, Research and Expert use progressively deeper planner budgets and wider `mode_chat_agent_profile()` fan-out. Quick still runs the orchestrator.

---

8. Inference layer (`eli/cognition`)

- **`inference_broker.py`**: the canonical inference path, `broker.infer()`.
- **`gguf_inference.py`**: the llama-cpp wrapper: live runtime parameters, `chat_completion()`, runtime snapshot publishing.
- **`model_identity.py`, `model_output_tokens.py`**: model family and chat template are read from GGUF metadata, not from the file name; special tokens are stripped per family.
- **`reasoning_modes.py`**: `quick`, `chain_of_thought`, `self_consistency`, `tree_of_thoughts`, `constitutional_ai` (private strategies; the raw reasoning is not shown).
- **`context_synthesiser.py`, `user_info_builder.py`**: prompt and context assembly.
- **`context_budget.py`**: how much of the window memory may use. Output reserve is an eighth of the window; on a recall question the memory floor is a third of it.

- **`llm_intent.py`**: LLM fallback for ambiguous routing. Decoding is constrained by a GBNF grammar built from the live action catalogue, so the model can only name a real capability and emit well-formed JSON. Backends without grammar support fall back to free text.
- Budgeting lives in `engine.py::_build_enhanced_system` and `core/startup_hardware_optimizer.py`.

---

9. Grounding and anti-confabulation (`eli/runtime`, `eli/core/grounding.py`)

- **`deterministic_grounding_gate.py`**: decides when an answer must be backed by evidence and renders self-reports and audits from live measurements. `render_action` is one explicit ordered pipeline of layers (`_stack`), not captured copies.
- **`grounding_escalation.py`**: a checkable factual turn with low grounding escalates (external fact → web agent, self or project fact → broad local fan-out) and hedges honestly if nothing grounds it. Offline (the default), a question about what does not change (a definition, history, who wrote a book) is answered from the model's own knowledge, the prompt saying the network is off; only one about what changes after a model is trained (news, prices, results, office-holders, release dates, this year or last, "the new one": `changes_over_time`) is hedged. Telling ELI something is not a lookup, but saying what an earlier question meant ("I'm talking about the newest movie", `clarifies_a_question`) is that question again; with a show or film being talked about or playing (`conversation_thread.extract_thread_topic` reads ELI's own "Playing: …" line, never a placeholder like "something else"), "the new one" of it is looked up, and a search asked for with no subject of its own ("do a web search, you should be smart enough") searches what the talk is about, never the remark. Env: `ELI_GROUNDING_ESCALATION`.
- **`evidence_ledger.py`, `evidence_arbitration.py`**: what counts as evidence, where it is recorded (the ledger also records every executed action), and conflict resolution.
- **`persistence_gate.py`**: keeps internal report dumps out of memory.
- **`response_contracts.py`, `response_policy.py`, `final_response_provider.py`, `user_visible_response_surface.py`**: shape and clean the user-visible answer.
- **`truth_report.py`, `eli_identity_audit.py`, `control_contracts.py`**: runtime truth, identity grounding, control-action contracts.
- **`cognition/output_governor.py`, `persona_hygiene.py`**: the final governance pass, including removal of false diagnoses when retrieval demonstrably ran.

The known weak seam is internal state leaking into spoken output on the plain CHAT path (identity confabulation, telemetry dumps). It is defended, not closed.

---

10. Execution layer (`eli/execution/executor_enhanced.py`)

- `execute(action, args) -> dict`. The executor's supported list holds **207 actions**; the manifest declares **230 capabilities**, of which **189 are routable** and 207 are in the supported list.
- **Fast path** (`engine.py`): deterministic OS, media, status and job actions (`VOLUME`, `MEDIA_CONTROL`, `NEXT_MEDIA`, `OPEN_APP`, `DATE`, `SHELL_EXEC`, `ANALYZE_IMAGE`, `CHECK_JOB`, `BACKGROUND_JOBS`, …) return the executor result verbatim.
- **Calendar and reminders** (`runtime/agenda.py`): ELI's own, stored locally. `ADD_EVENT` (also moves an event: "i meant 8pm"), `LIST_EVENTS`, `REMOVE_EVENT`, and `SET_ALARM` for "remind me …". A notifier thread delivers what is due; the next 36 hours go into every prompt as facts, next to the clock times worked out from the user's message.
- Action families: media and OS control, files and documents (`SUMMARIZE_FILE`, `ANALYZE_PDF[_FOLDER]` including per-file mode and saving into memory, `CREATE_DOCUMENT`), web (`WEB_SEARCH`), news, weather, memory (`MEMORY_STORE`, `MEMORY_RECALL`), code (`CODE_SOLVE`, `GENERATE_SCRIPT`), background jobs, self and runtime reports (`RUNTIME_STATUS`, `SELF_REPORT`, `RUNTIME_AUDIT`, `EXPLAIN_*`), vision, image generation, scheduling.
- **Background jobs** (`runtime/background_tasks.py`): `submit/get/list`. Heavy PDF-folder analysis backgrounds itself; `CHECK_JOB` returns the real result.

- **YouTube playback** lives in `eli/integrations/media/youtube_playback.py`; the executor delegates to it.

Plugins (`eli/plugins`)

Bundled: `calendar`, `document_reader`, `media`, `notes`, `pomodoro`, `system_stats`, `tts`, `weather`, `web`, `web_automation`. State is in `config/plugins_state.json` (gitignored). Custom agents and plugins are trust-gated (§ plugin table below).

---

11. Memory subsystem (`eli/memory`)

- **`memory.py`** (`Memory`, `get_memory()`): long-term store over SQLite and FTS5, plus failure, observation and habit logging.
- **`policy.py`**: the storage policy. Every row carries an origin (`user_said`, `eli_said`, `telemetry`, `news`, `tool`); repeats are merged (`seen_count`); a forgetting curve scales weight by importance, origin and reinforcement; faded derived rows move to `memories_archive`. A daily `run_upkeep()` does the merging, decay, archiving, identity merge and learning-table tidy. Full detail in `memory.md`.
- **`vector_store.py`**: FAISS index at `artifacts/vectors/index.faiss`; embedder `models/embeddings/nomic-embed-text-v1.5.Q4_K_M.gguf` *(runtime)*.
- **Knowledge graph**: `kg_entities` and `kg_relations` (FTS5-backed) in the user database.
- **Working memory** (`cognition/working_memory.py`): session-pinned facts with dates preserved, restored across sessions.
- **Personal memory**: `runtime/personal_memory_*`, `persona_updater`.

Four SQLite files under `artifacts/db/` *(runtime)*: `user.sqlite3` (memory, conversation, knowledge graph, learning tables), `agent.sqlite3` (self-improvement records and agent metrics), `system_index.sqlite3` (OS index) and `coding_memory.sqlite3` (coding bug fixes). The blank `user.sqlite3` template holds 28 tables plus three FTS5 indexes: `memories`, `memories_archive`, `memory_meta`, `memory_lineage`, `semantic`, `conversation_turns`, `conversations`, `session_summaries`, `kg_entities`, `kg_relations`, `recall_log`, `runtime_events`, `learning_replay`, `observations`, `habits`, `habit_events`, `habit_rules`, `user_patterns`, `user_model`, `eli_stances`, `belief_revisions`, `corrections`, `failures`, `error_tracking`, `improvements`, `capability_proposals`, `news_articles`, `news_reflections`. Self-improvement records (`improvements`, `failures`, `corrections`, `capability_proposals`, `error_tracking`) are written to `agent.sqlite3`; the copies in the user database are empty. Recall is hybrid: keyword, FTS5, FAISS, knowledge graph and recent conversation, merged and reranked (§7).

---

12. Perception (`eli/perception`)

- **Vision**: `vision.py` (model-agnostic vision-language, Moondream and Qwen2.5-VL hot-swap; the multimodal encoder is forced to CPU to avoid a CUDA clip segfault), `analyze_image.py`, `ambient_vision.py`, `screen_locator.py`, `gaze_engine.py`, `analyze_csv.py`, `analyze_pdfs.py`.
- **STT**: `local_whisper_stt.py` (faster-whisper, offline when the Net switch is off), `audio_stt.py`, `voice_worker_streaming.py`. Default wake phrases: “computer”, “hey computer”, “eli”, “hey eli” (`wakeword.py`); a music-bleed filter and an echo gate sit in front of the transcript.
- **TTS**: `tts_router.py` (Piper voices), `tts_xtts.py` (optional neural voices).
- **OS**: `os_controller.py` (app, window, keyboard, mouse), `ui_tree.py`, `desktop_capabilities.py`.

---

13. Persona (`eli/cognition/persona*`)

- `persona.txt` (authored) and `persona.auto.txt` (overlay) hold the voice.
- `persona_updater.py` updates the overlay, knowledge graph and user profile each turn.

- persona.py, persona_values.py, persona_status.py, persona_hygiene.py.
- tone_analyzer.py and tone_adaptor.py read the user's tone; emotion_timeline.py keeps a history that lets ELI notice a sustained mood.
- Generation injects a budget-trimmed persona and situation brief (engine.py::_build_enhanced_system); a live-runtime fact is injected for model and identity questions so ELI reports the loaded model.

---

14. Daemons and background loops

- **Proactive daemon** (planning/proactive_daemon.py): pattern signals (time habit, topic focus, recurring errors, active project); runs an autonomy tick every 30 minutes.
- **Self-improvement loop** (runtime/self_improvement.py): started at boot, analyses failures immediately and then every 24 hours. Repair by the coding agent is opt-in and bounded. A failure is stored with a capsule (input, versions, model, error class), and a candidate patch is tried on a hard-linked copy of the tree and compared with the same tests on the untouched tree: only a candidate that fixes something and breaks nothing is applied, every candidate stays in the archive (code_patches) with its verdict and cost, and the tests, the evaluator and the safety gates cannot be patched by a candidate. ELI_SELFPATCH_UNPROVEN=1 restores applying without proof.
- **Habit scheduler** (planning/habits_scheduler.py, habits.py).
- **Scheduler** (kernel/scheduler.py); **background tasks** (runtime/background_tasks.py).
- **Code monitor** (runtime/code_monitor.py); **ambient vision loop**.
- **World event bus** (world/world_event_bus.py).
- **Memory upkeep**: once a day, started asynchronously at engine init and after responses.

---

15. Learning and self-training (eli/learning)

A LoRA fine-tuning pipeline: bootstrap_phi3_base.py, base_model_resolver.py, dataset_builder.py, dataset_filters.py, merge_reviewed_datasets.py, export_trainable_dataset.py, training_preflight.py, lora_trainer.py, lora_trainer_guard.py, lora_eval.py, plus target_registry.py (operator-declared targets, any model family) and review_queue.py (the human review gate).

---

16. EliWorld (eli/world, the EliWorld tab, kernel/world_model.py)

An internal world the agent inhabits. world_event_bus.py receives runtime events; local_world_bridge.py bridges to the model; snapshots, ledger and journal are under artifacts/world/. Experimental.

---

17. Core infrastructure (eli/core)

- **netguard.py**: offline by default: guarded_urlopen, a process-wide socket guard (fail-closed on non-loopback while offline), and allow_network() for deliberate user-initiated fetches.
- **paths.py, portable_paths.py, legacy_paths.py, db_paths.py**: path resolution (dev tree vs platform directories, ELI_* overrides, read-only frozen bundles).
- **runtime_settings.py, config.py**: settings load, merge and heal; DEFAULTS are clean (offline, no model, wizard on). config/settings.json is per user and gitignored, seeded from config/templates/settings.template.json.
- **hardware_profile.py, startup_hardware_optimizer.py**: detect GPU and VRAM and pick context, GPU layers and batch (one fit calculation shared by the loader and the startup dialog).
- **model_download.py**: curated GGUF downloader (7 catalogue entries, resumable, magic and size validated, netguard-gated). Install-time menu only.
- **grounding.py**: shared grounding helpers.

18. GUI (`eli/gui`)

PySide6 (LGPL). `eli_pro_audio_gui_v2_0.py` (main window, 13.1k lines), `app.py` (boot), `labs_tab.py` (Labs, 5.7k lines), `panels/` (`startup.py`: StartupModelSelectionDialog, FirstBootWizard and the hardware tuning dock; `settings.py`, `agent_wizard.py`, `permission_dialog.py`), `tabs/` (`eli_world_tab.py`, `experimental_tab.py`, `marketplace_tab.py`, `tasks_tab.py`, `training_tab.py`), `docks/` (operator console, proactive dock), `widgets/` (animated face, voice downloader, Ollama selector). `qt_compat.py` allows a PyQt fallback from source.

---

19. On-disk layout (`artifacts/`, runtime)

```
artifacts/
├── db/{user,agent,system_index,coding_memory}.sqlite3
├── vectors/index.faiss            # semantic index
├── conversations/                 # conversation exports
├── documents/ , scripts/          # generated artifacts
├── runtime/                       # runtime state and capability inventory
├── runtime_snapshot.json          # live model/runtime truth
├── world/                         # snapshots, ledger, journal
└── analyze_image_*/               # vision outputs
config/  settings.json (gitignored) · templates/ · plugins_state.json · voices/ …
models/  <your>.gguf · embeddings/ · whisper/ · image/
tts_piper/  Piper voice files shipped with the repo
```

---

20. Known seams and weak points

1. **Large files.** `engine.py`, `executor_enhanced.py`, the main GUI file and the router hold about a third of the code. The executor is an action ladder with a high regression surface.
2. **Internal state → spoken output.** The recurring failure class. The grounding gate and escalation defend it; they do not close it on the plain CHAT path.
3. **Swallowed errors.** 162 handlers are a bare `except: pass` and 4,392 catch `Exception`, most of them logging at debug level. A ratchet test (`tests/claims/test_no_silent_swallow.py`) stops the silent count rising.
4. **Latency versus model size.** On an 8 GB GPU a 24B model at Q5 offloads few layers and answers in minutes; a 7–9B Q4 is the sweet spot. The startup dialog shows the real fit.
5. **Two GUI monoliths.** `eli/gui/eli_pro_audio_gui_v2_0.py` and `eli/gui/labs_tab.py` are single files of 13.1k and 5.7k lines.

---

21. Where to make a change

To change…	Edit…
how input is routed to an action	`eli/execution/router_enhanced.py`
what an action does	`eli/execution/executor_enhanced.py`
which agents run and how grounded	`eli/cognition/agent_bus.py`
the 12-stage orchestration	`eli/cognition/orchestrator.py`
prompt and persona budget, the engine spine	`eli/kernel/engine.py`

To change…	Edit…
anti-confabulation	`eli/runtime/grounding_escalation.py`, `deterministic_grounding_gate.py`
final answer shaping	`eli/runtime/*response*`, `cognition/output_governor.py`
memory store and recall	`eli/memory/memory.py`, `policy.py`, `retrieval.py`, `vector_store.py`
model loading and inference	`eli/cognition/gguf_inference.py`, `inference_broker.py`
offline and network policy	`eli/core/netguard.py`
paths, settings, model download	`eli/core/{paths,runtime_settings,model_download}.py`
the GUI	`eli/gui/eli_pro_audio_gui_v2_0.py`, `eli/gui/panels/`, `eli/gui/tabs/`
evaluation and regression	`tools/eval/`

Plugins, agent specifications and training surfaces

`eli/plugins/`: a loader plus a gated marketplace client

Module	Lines	Role
`manager.py`	708	discovery, enable and disable, install and uninstall
`permissions.py`	395	capability vocabulary, consent decisions, grants, audit ledger
`manifest.py`	302	manifest schema and static capability verification against source
`integrity.py`	291	sha256 pinning, ed25519 signatures, operator-trusted publishers
`security_scan.py`	547	multi-engine malware scanner
`marketplace.py`	793	federated registries, preview and install, licence keys
`mcp.py`	481	MCP server config, runtime preflight, handshake verification

`_plugins_dir()` resolves through `paths.plugins_dir()`, and `_plugin_search_dirs()` scans bundled built-ins and user installs, so a downloaded plugin is never written into a read-only installation.

`eli/cognition/`: agents get a specification

Module	Lines	Role
`agent_spec.py`	436	`AgentSpec`: objective, prompt, triggers, success criteria, examples

Module	Lines	Role
agent_trust.py	262	path-keyed trust with provenance, scanning, revocation

agent_bus.SpecAgent runs a spec against the local model and scores the output against the spec's own criteria. Spec agents execute no arbitrary code and bypass the trust chain.

GUI surfaces

tabs/training_tab.py (Labs ▸ Training), tabs/marketplace_tab.py (Settings ▸ Marketplace), panels/permission_dialog.py (consent dialog and worker-to-GUI-thread bridge).

Paths

paths.learning_dir() joins models_dir(), voices_dir() and plugins_dir(). The rule: runtime state never lives in the installation, because project_root() is a read-only mount in a packaged build.